UNIVERSIDAD DEL VALLE DE GUATEMALA

COMPUTACIÓN PARALELA Y DISTRIBUIDA

Sección 20

Ing. Santiago Solorzano Padilla



Proyecto 2

Búsqueda exhaustiva de llaves DES con OpenMPI: análisis de speedup y rendimiento

Mia Fuentes, 23775

Diego Rosales, 23258

Diego López, 23242

21 de octubre de 2026

Investigación del algoritmo DES

Concepto general y tamaño de bloque

El Data Encryption Standard (DES) es un algoritmo de cifrado simétrico por bloques: la misma llave secreta permite cifrar y descifrar. Transforma cada bloque de texto de 64 bits (8 bytes) en otro bloque de 64 bits. Su llave se representa en 64 bits, aunque únicamente 56 son efectivos; los ocho bits restantes se destinan a la paridad impar, uno por cada byte. La estructura interna corresponde a una red Feistel de 16 rondas (NIST, 1999).

1. Permutación inicial (IP)

Antes de procesar el bloque, DES aplica la permutación inicial IP. Se trata de una reordenación fija de las 64 posiciones: no añade ni elimina bits y tampoco utiliza la llave. A continuación, el resultado se divide en dos mitades de 32 bits, L_0 y R_0. Estas mitades son las entradas de la primera ronda Feistel.

2. Las 16 rondas Feistel

En cada ronda i, para i = 1, …, 16, DES combina la mitad derecha previa con una subllave de 48 bits K_i mediante la función F. La mitad izquierda de la nueva ronda recibe la mitad derecha previa y la derecha nueva resulta del XOR entre la izquierda previa y la salida de F:

$$L_i = R_{i-1} \qquad\qquad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

La función F se compone, exactamente en este orden, de cuatro operaciones: (a) expansión E, que transforma los 32 bits de R_{i-1} en 48 bits al repetir determinadas posiciones; (b) XOR bit a bit con la subllave K_i de 48 bits; (c) sustitución mediante ocho cajas S, cada una de las cuales convierte 6 bits en 4 bits, reduciendo los 48 bits a 32; y (d) permutación P, que reorganiza esas 32 posiciones para dispersar el resultado antes del XOR de la ronda. Las cajas S introducen la transformación no lineal del algoritmo.

3. Permutación final (IP^{-1})

Después de la decimosexta ronda, las mitades se concatenan en orden $R_{16} \parallel L_{16}$. A ese bloque de 64 bits se le aplica IP^{-1}, la permutación inversa de IP. El resultado es el bloque cifrado de 64 bits. Las permutaciones inicial y final son operaciones distintas de las sustituciones realizadas dentro de F.

4. Generación de las 16 subllaves

La programación de llaves comienza con una llave DES de 64 bits, incluidos sus bits de paridad. PC-1 selecciona y permuta 56 bits, descartando los ocho bits de paridad; el resultado se divide en C_0 y D_0, de 28 bits cada uno. En cada ronda se efectúa una rotación circular a la izquierda, de uno o dos bits, tanto en C como en D. Luego PC-2 selecciona y permuta 48 de los 56 bits concatenados para formar la subllave K_i.

Las rotaciones de las rondas 1 a 16 son: 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2 y 1 posiciones. Se obtienen así K_1, K_2, …, K_{16}, cada una de 48 bits. PC-1 y PC-2 forman parte del algoritmo de generación de subllaves; no deben confundirse con IP, IP^{-1} ni la conversión previa de un entero decimal a los ocho bytes de la llave.

5. Descifrado: mismo algoritmo, subllaves inversas

DES descifra aplicando la misma permutación inicial, las mismas 16 rondas Feistel y la misma permutación final. La diferencia es el orden de las subllaves: K_{16}, K_{15}, …, K_1. La estructura Feistel permite recuperar el bloque original sin invertir individualmente la función F ni las cajas S. Con la llave correcta, el bloque descifrado es idéntico al bloque de entrada previo al cifrado.

Resumen técnico de transformaciones DES

Operación	Entrada → salida	Finalidad
IP	64 → 64 bits	Reordenar bits antes de las rondas.
E	32 → 48 bits	Expandir la mitad derecha para mezclar con K_i.
XOR con K_i	48 ⊕ 48 bits	Combinar datos con la subllave de la ronda.
Ocho S-boxes	48 → 32 bits	Sustitución no lineal: 6 → 4 bits por caja.
P	32 → 32 bits	Reorganizar salida de las S-boxes.
IP^{-1}	64 → 64 bits	Permutar el bloque final a su orden de salida.
PC-1	64 → 56 bits	Descartar paridad y seleccionar la llave efectiva.
PC-2	56 → 48 bits	Formar cada subllave K_i después de rotar C y D.

Diagrama de flujo de DES

La figura resume la entrada de 64 bits, la permutación IP, las 16 rondas con E, XOR, cajas S y P, el intercambio final de mitades y la permutación IP^{-1}. La generación de las subllaves alimenta cada ronda.

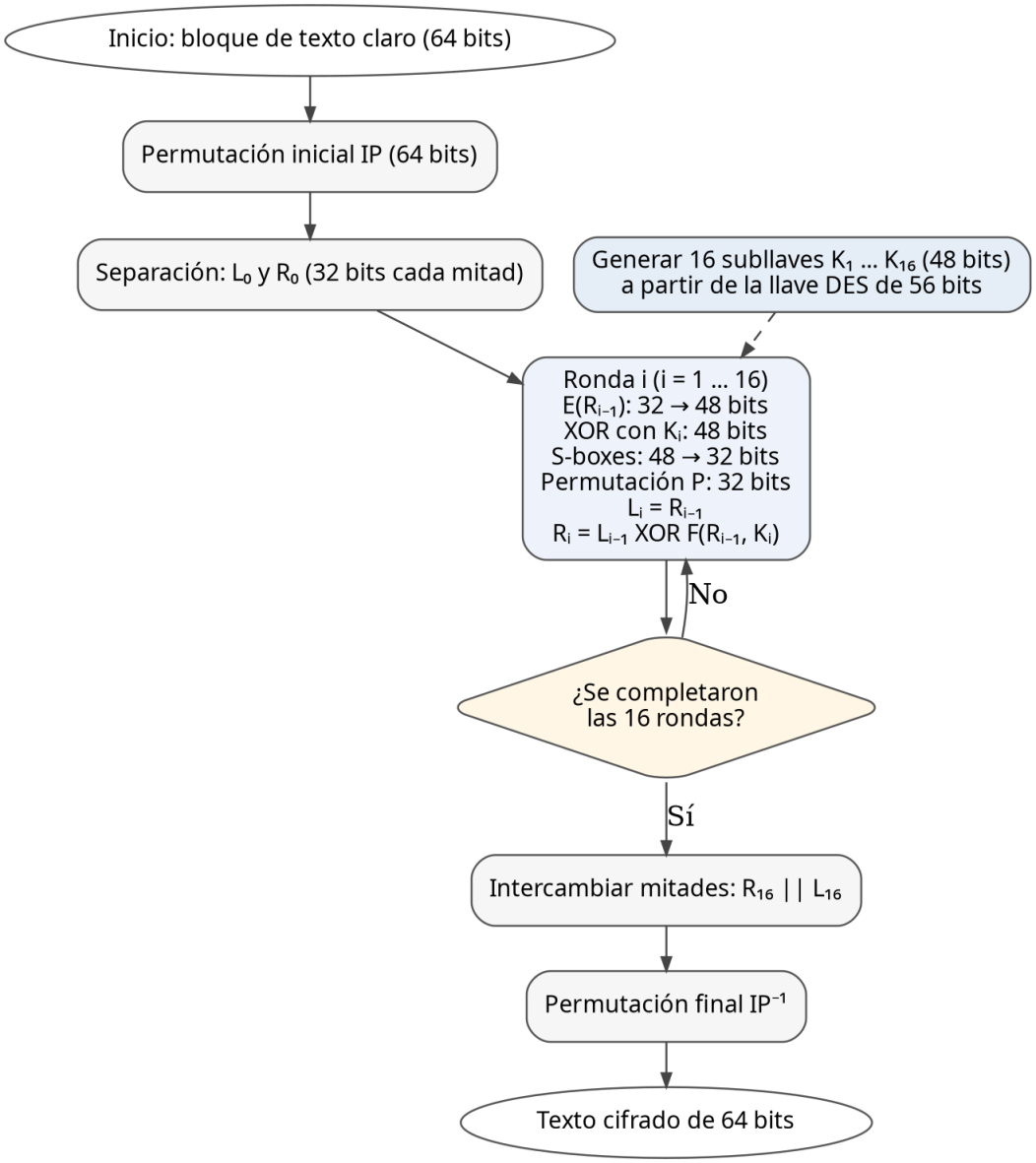


Figura 1. Diagrama de flujo de cifrado DES: 16 rondas Feistel y permutaciones.

Diagramas detallados de encrypt y decrypt

Conversión de la llave efectiva: 56 → 64 bits

Las funciones encrypt(long key, char *buf, int len) y decrypt(long key, char *buf, int len) requieren una llave DES con paridad. La función auxiliar des_key_to_bytes toma el entero no negativo inferior a 2^{56}, obtiene ocho grupos de siete bits y reserva el bit menos significativo de cada byte para ajustar la paridad impar. DES_set_odd_parity (OpenSSL) o des_setparity (libtirpc) completan los ocho bytes. Así se construye una representación de 64 bits con 56 bits de llave y ocho de paridad.

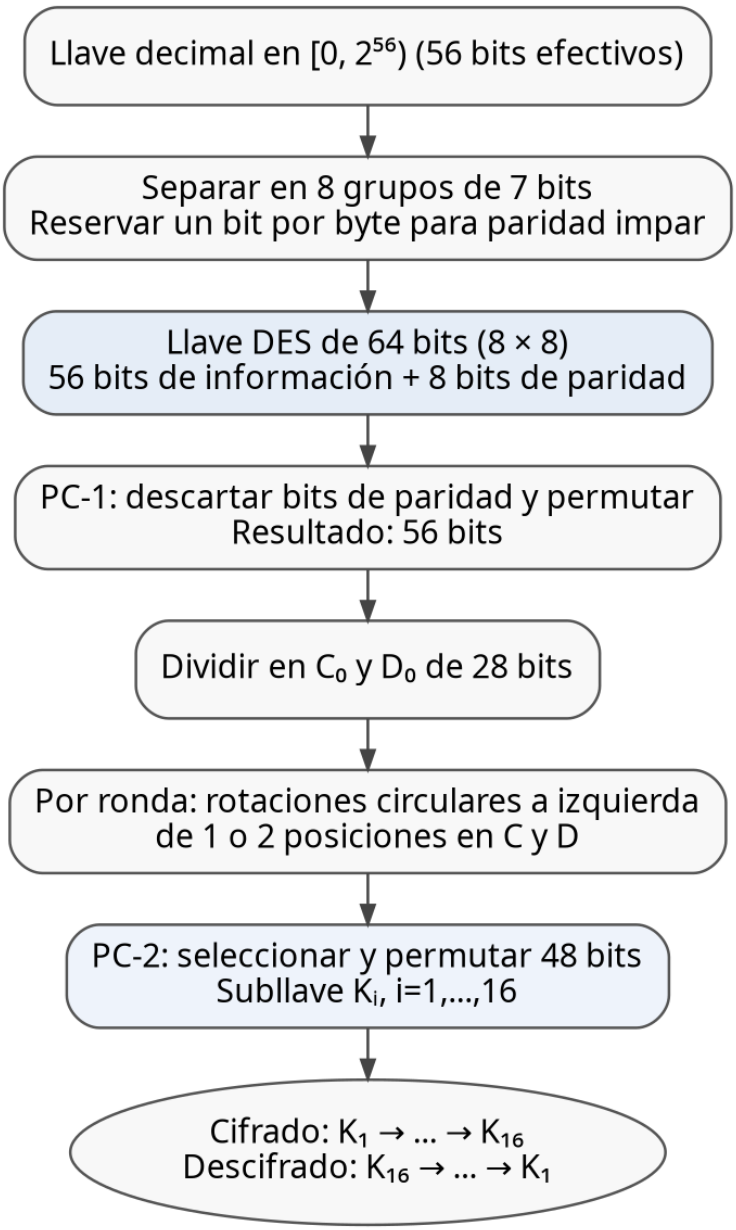


Figura 2. Conversión de 56 a 64 bits y generación de subllaves PC-1/PC-2.

Cifrado y descifrado ECB, bloque por bloque

Ambas funciones reciben un buffer binario y una longitud explícita len. El tamaño debe ser positivo y múltiplo de ocho; las funciones de transformación no agregan ni retiran relleno. Cuando el texto no cumple ese tamaño, antes de cifrar se aplica padding PKCS#7 y, después de descifrar, se valida y retira. En ECB se divide el buffer en B_1, B_2, …, B_n de ocho bytes y se procesa cada bloque de forma independiente con la misma llave (Dworkin, 2001).

$$C_i = DES_k(B_i) \qquad B_i = DES_k^{-1}(C_i)$$

El módulo des_crypto.c prepara una sola vez la llave y, con OpenSSL, su DES_key_schedule. Luego recorre i = 0, …, n − 1 y transforma los ocho bytes de cada bloque con DES_ecb_encrypt en modo cifrado o descifrado, escribiendo el resultado en la misma posición del buffer (in-place). Durante el cifrado se usan $K_1 \rightarrow K_{16}$; durante el descifrado, $K_{16} \rightarrow K_1$. El modo ECB no encadena bloques, por lo que bloques de texto iguales producen bloques cifrados iguales para la misma llave; se utiliza aquí con fines académicos.

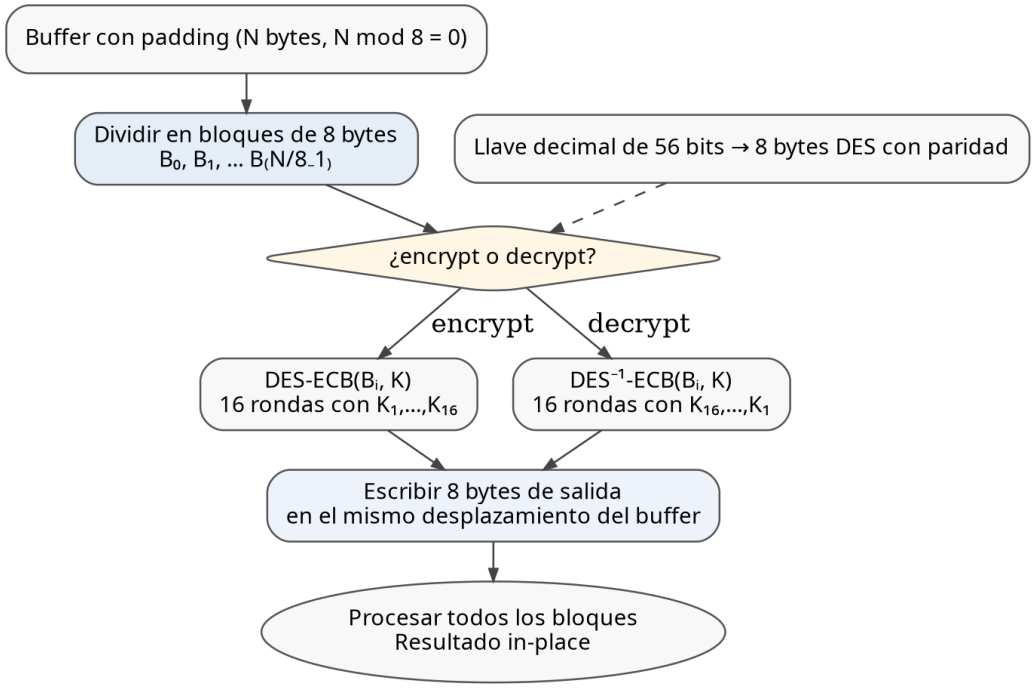


Figura 3. Flujo de encrypt/decrypt: misma llave, ECB por bloques de 8 bytes.

Compilación y ejecución de bruteforce_.c

Entorno

Las pruebas se realizaron en Ubuntu 26.04 sobre WSL 2, con GCC 15.2, OpenMPI 5.0.10 y OpenSSL 3.5.5, en un procesador Intel Core Ultra 7 255HX (8 CPU asignados a WSL). El código completo, los diagramas y los datos de las mediciones están en el repositorio del proyecto.

Compilación del programa original

El archivo entregado se compiló sin modificaciones, con el comando recomendado para libtirpc. La compilación falla porque el encabezado `<rpc/des_crypt.h>` ya no forma parte de la glibc moderna y no está disponible en este entorno:

```
$ mpicc bruteforce_original.c -o bruteforce_original -I/usr/include/tirpc -ltirpc
bruteforce_original.c:6:10: fatal error: rpc/des_crypt.h: No such file or directory
    6 | #include <rpc/des_crypt.h>
      |          ^~~~~~~~~~~~~~~~~
compilation terminated.
```

Por esta razón el equipo escribió un módulo de cifrado propio, `des/des_crypto.c`. El `Makefile` intenta enlazar `ecb_crypt` y `des_setparity` de libtirpc y, si no están disponibles, compila el módulo con la interfaz DES de OpenSSL (`DES_ecb_encrypt`). Los dos backends trabajan en modo ECB, con la misma conversión de llave y el mismo formato de salida.

Defectos del programa original que se corrigieron

La lectura del código original mostró varios defectos que invalidan cualquier medición si no se corrigen. La Tabla 1 los resume junto con su corrección.

Tabla 1. Defectos del programa original y su corrección.

Defecto	Consecuencia	Corrección
`ecb_crypt` recibe 16 como longitud	Solo se cifran los primeros 16 bytes. Con el texto de 350 palabras la frase clave queda legible y `tryKey` aceptaría cualquier llave.	`encrypt` y `decrypt` procesan los `len` bytes completos, bloque por bloque.
El texto no es múltiplo de 8 bytes	DES opera con bloques de 8 bytes.	Padding PKCS#7 antes de cifrar; se valida y retira al descifrar.
`0xFE << i*8` con `int` de 32 bits	Desbordamiento (comportamiento indefinido) para $i \geq 4$: la llave se arma mal.	`des_key_to_bytes` arma los ocho bytes con aritmética de 64 bits sin signo.
Llave fija de 2^{54}	Con 3 procesos la búsqueda tardaría años.	La llave de cifrado, el archivo cifrado, la frase y el límite `upper` se reciben por línea de comandos.
`<rpc/des_crypt.h>` no disponible	El programa no compila.	Módulo con dos backends (libtirpc u OpenSSL) y `Makefile` que elige el disponible.
Límites del bloque: `myupper` inclusivo con `i < myupper`	La última llave de cada bloque nunca se prueba.	El ciclo recorre `i <= myupper` y el último bloque llega hasta `upper - 1`.
`MPI_Irecv` escribe en la misma variable `found` que el proceso modifica	El estándar MPI no permite acceder al buffer de una recepción pendiente.	La recepción usa un buffer propio (`received`).
`MPI_Wait` del proceso 0 espera siempre un aviso	Si la llave no está en el rango, el programa nunca termina.	`MPI_Allreduce` con `MPI_MAXLOC`: todos saben si hubo llave; si no, se cancela la recepción.
Arreglo `temp[len+1]` (VLA) en cada `tryKey`	Reserva en la pila en cada llave; riesgo con textos grandes.	Buffer dinámico reservado una sola vez.
Texto, frase y llave dentro del código	No se pueden automatizar las mediciones.	Argumentos de línea de comandos (ver más abajo).

Compilación del programa final

El `Makefile` de la raíz compila `bruteforce_.c` junto con `des/des_crypto.c` y las utilidades de `des/`. Acepta `TEST_EVERY=K` para elegir cada cuántas llaves se revisa el aviso de terminación. En este equipo se usó OpenSSL como backend y la compilación termina sin advertencias:

```
$ make
mpicc -std=c11 -O2 -Wall -Wextra -Ides -DTEST_EVERY=1 -DDES_USE_OPENSSL
      -Wno-deprecated-declarations bruteforce_.c des/des_crypto.c
      -o bruteforce -lcrypto
```

Uso y ejecución

El programa recibe el archivo cifrado, la frase clave, el límite superior de la búsqueda (`upper`) y, de forma opcional, el tipo de reparto. Busca llaves en [0, `upper`) con $1 \leq$ `upper` $\leq 2^{56}$:

```
mpirun -np N ./bruteforce <cifrado.des> "<frase>" <upper> [bloques|intercalado]
```

Con la llave 70 000 y `upper` = 120 000 se obtuvo lo siguiente (la salida muestra el rango de cada proceso, quién encontró la llave, el texto recuperado y la línea `RESULTADO`, con N, la llave, el proceso que la encontró y los segundos, que lee el script de mediciones):

```
$ mpirun -np 1 ./bruteforce cifrado.des " los mismos del proyecto 1 " 120000
Proceso 0: rango [0, 119999] paso 1
Proceso 0 encontro la llave
Llave = 70000

La computación paralela permite resolver problemas grandes mediante la cooperación ...
RESULTADO,1,70000,0,1.219481

$ mpirun -np 3 ./bruteforce cifrado.des " los mismos del proyecto 1 " 120000
Proceso 0: rango [0, 39999] paso 1
Proceso 1: rango [40000, 79999] paso 1
Proceso 2: rango [80000, 119999] paso 1
Proceso 1 encontro la llave
Llave = 70000
RESULTADO,3,70000,1,0.547662

$ mpirun -np 3 ./bruteforce cifrado.des " los mismos del proyecto 1 " 120000 intercalado
Proceso 0: rango [0, 119999] paso 3
Proceso 1: rango [1, 119999] paso 3
Proceso 2: rango [2, 119999] paso 3
Proceso 1 encontro la llave
Llave = 70000
RESULTADO,3,70000,1,0.413773
```

En los tres casos se recupera la llave 70 000 y el texto original. Con un solo proceso hacen falta 70 001 intentos. Con tres procesos por bloques, la llave cae en el bloque del proceso 1 (40 000 a 79 999) y este la encuentra tras 30 001 intentos. Con el reparto intercalado la encuentra también el proceso 1 (70 000 mod 3 = 1), tras 23 334 intentos, porque cada proceso recorre una de cada tres llaves.

Cifrado del texto de prueba

Para las pruebas se preparó un texto en español de **350 palabras** (el enunciado pide 350 ± 15) y 2 489 bytes. Contiene exactamente una vez la frase clave `los mismos del proyecto 1` (27 bytes, con un espacio al inicio y otro al final), en el segundo párrafo: «…Nuestro equipo decidió conservar los mismos del proyecto 1 para organizar la implementación…». En el texto sin cifrar, la frase comienza en el byte 897. El texto abre así: «La computación paralela permite resolver problemas grandes mediante la cooperación de varios procesos que trabajan al mismo tiempo…».

Elección de la llave

La llave privada es un entero en $[0, 2^{56})$. Para este ejemplo y para casi todos los diagramas se eligió la llave **70 000**. En las mediciones se usan otras llaves, porque el objetivo del análisis es observar cómo cambia el tiempo con la llave. Recorrer las 2^{56} llaves tardaría unos 40 000 años con un proceso (≈ 57 000 llaves por segundo por proceso en el equipo usado), así que las llaves de prueba son pequeñas y la búsqueda se acota con `upper`.

Proceso de cifrado

La utilidad `des/encrypt_file` valida que el texto tenga entre 335 y 365 palabras, agrega padding PKCS#7 y cifra con DES en modo ECB. El resultado se guarda como archivo binario, porque contiene bytes nulos y no puede imprimirse como cadena; el archivo no almacena la llave ni la frase.

```
$ des/encrypt_file texto_350_palabras.txt 70000 cifrado.des
Cifrado completado.
Palabras: 350 (rango requerido: 335-365).
Llave DES efectiva (decimal): 70000.
Longitud original: 2489 bytes.
Longitud cifrada con PKCS#7: 2496 bytes.
Archivo binario: cifrado.des

$ xxd cifrado.des | head -3
00000000: af3f 5fb6 e8fd dd0d a030 8e7e b9d5 5ed8  .?_......0.~..^.
00000010: e3dc d61c 7a99 37c8 3ac0 c617 b19b 1fd6  ....z.7.:.......
00000020: 0a3c e0de b5e4 b05d abe2 7a3a c723 b99e  .<.....]..z:.#..
```

Tabla 2. Resumen del texto cifrado.

Propiedad	Valor
Palabras	350
Longitud del texto	2 489 bytes
Padding PKCS#7	7 bytes con valor 0x07 (2 489 mod 8 = 1)
Longitud cifrada	2 496 bytes = 312 bloques de 8 bytes
Llave	70 000 (56 bits efectivos)
Frase clave	« los mismos del proyecto 1 » (27 bytes)

Para comprobar el cifrado, `des/decrypt_file` descifra el archivo con la llave correcta, valida y retira el padding y reproduce el texto original; la comparación byte a byte (`cmp`) no encuentra diferencias. Con una llave incorrecta se obtienen bytes sin sentido y la frase no aparece.

Nota: en la copia de trabajo en Windows, con finales de línea CRLF, el mismo texto mide 2 500 bytes (2 504 con padding, 313 bloques). Las mediciones (sección «Mediciones de tiempo y speedup») se hicieron con esa copia; la diferencia en el costo por llave es de 0.3 %.

Funcionamiento de tryKey, memcpy y strstr

`tryKey(key, ciph, len)` se ejecuta una vez por cada llave candidata, por lo que su costo es el costo unitario de toda la búsqueda. Internamente usa `decrypt`, `memcpy` y `strstr`; las dos últimas se explican por separado más adelante.

tryKey

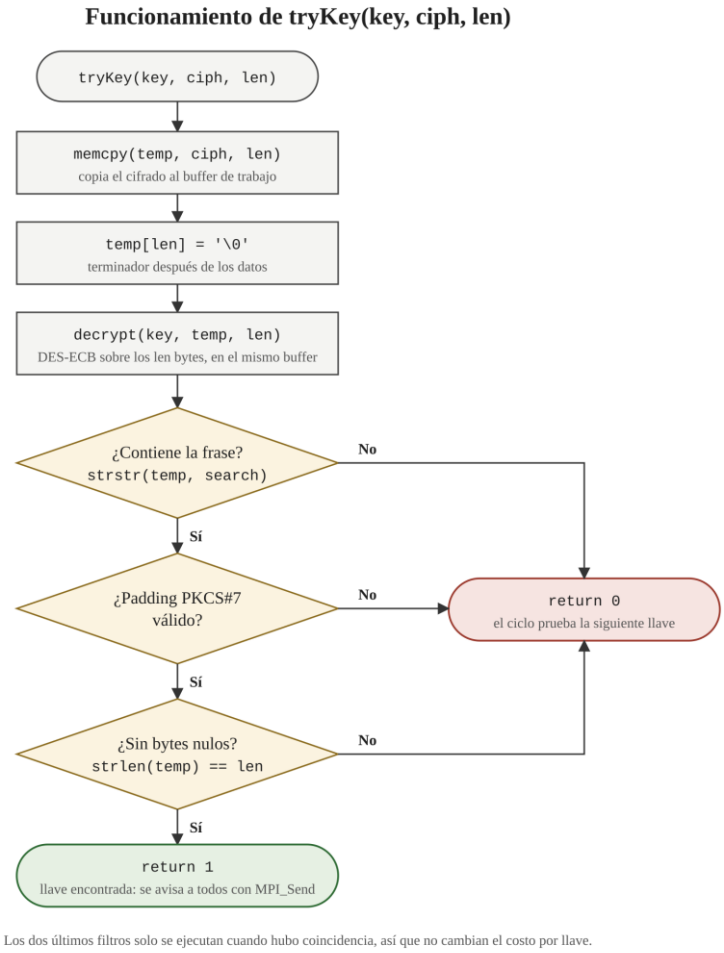


Figura 4. Funcionamiento de tryKey(key, ciph, len).

`tryKey` sigue cinco pasos:

1. `memcpy` copia el cifrado (`ciph`) a un buffer de trabajo `temp` y se escribe un `\0` después de los datos.
2. `decrypt` descifra `temp` en el mismo buffer con la llave candidata.
3. `strstr` busca la frase clave en el resultado. Si no aparece, la llave se descarta y `tryKey` devuelve 0.
4. Si la frase aparece, se aplican dos filtros: el padding PKCS#7 debe ser válido y el texto no debe contener bytes nulos. Si alguno falla, la llave se descarta.
5. Si todo se cumple, devuelve 1 y el proceso avisa a los demás.

```
int tryKey(long key, char *ciph, int len){
  memcpy(temp, ciph, len);
  temp[len] = 0;
  decrypt(key, temp, len);
  if(strstr(temp, search) == NULL)
    return 0;
  // descarta falsos positivos: el padding debe ser valido y no debe haber bytes nulos
  size_t plain_len;
  return des_pkcs7_unpad((unsigned char *)temp, len, &plain_len) == 0
      && strlen(temp) == (size_t)len;
}
```

Los dos últimos filtros solo se ejecutan cuando `strstr` encontró la frase, así que no cambian el costo por llave. Con una llave incorrecta el descifrado es texto aleatorio y la probabilidad de que contenga una frase de 27 bytes es despreciable. El costo de `tryKey` lo domina `decrypt`: 312 bloques DES por llave. En el equipo de medición una llamada completa tarda ≈ 17.5 µs, es decir, ≈ 57 000 llaves por segundo por proceso.

memcpy

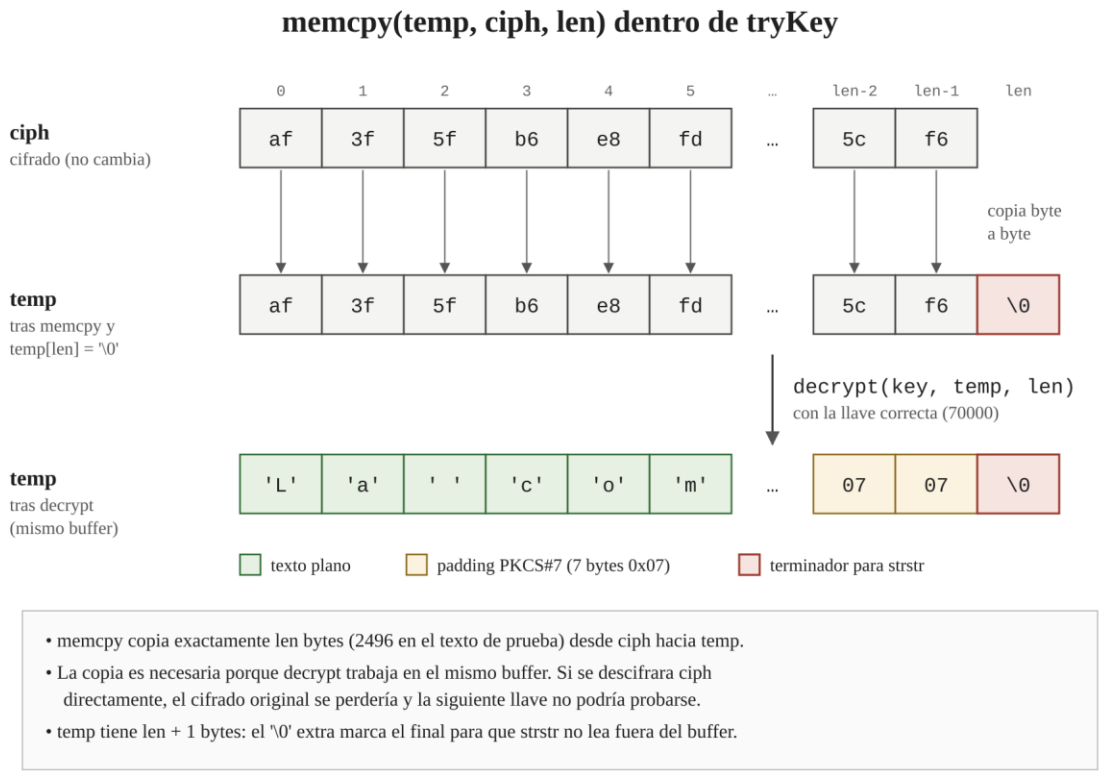


Figura 5. memcpy(temp, ciph, len) dentro de tryKey.

`memcpy(temp, ciph, len)` copia exactamente `len` bytes, byte a byte, desde `ciph` hacia `temp`. La copia es necesaria porque `decrypt` trabaja en el mismo buffer (in-place): si se descifrara `ciph` directamente, el cifrado original se perdería y la siguiente llave ya no tendría qué probar.

A diferencia de `strcpy`, `memcpy` no se detiene en un byte nulo. Esto es indispensable porque un texto cifrado contiene bytes `0x00` con frecuencia aproximada de uno por cada 256 bytes. El buffer `temp` mide `len + 1` bytes: el byte adicional recibe el terminador `\0`, que evita que `strstr` lea fuera del buffer. `temp` se reserva una sola vez al inicio del programa, no en cada llave.

strstr

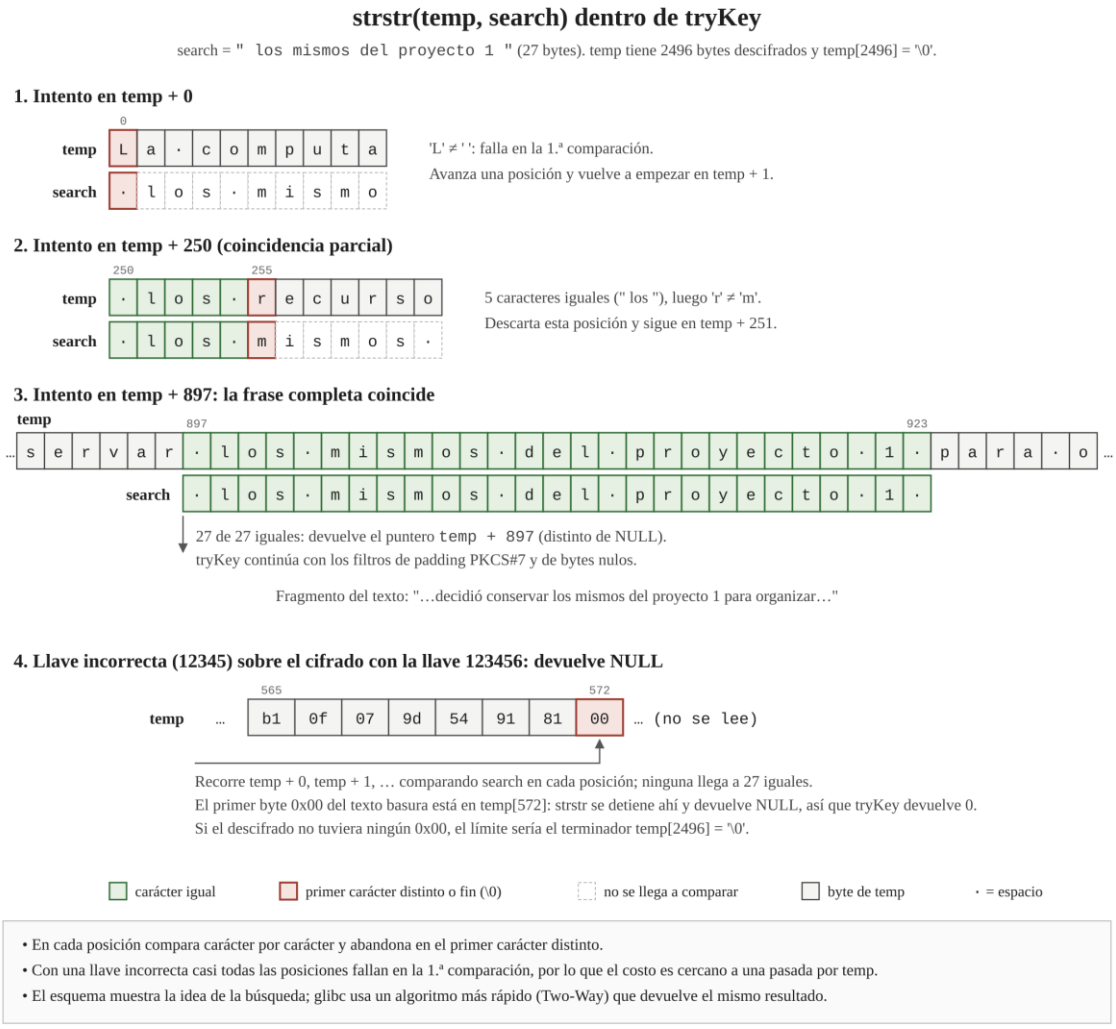


Figura 6. strstr(temp, search) dentro de tryKey.

`strstr(temp, search)` devuelve un puntero a la primera aparición de la frase `search` dentro de `temp`, o `NULL` si no aparece. Conceptualmente prueba cada posición de `temp`: compara la frase carácter por carácter y abandona en el primer carácter distinto. Con una llave incorrecta casi todas las posiciones fallan en la primera comparación, así que el costo es cercano a una pasada por `temp`. Además, `strstr` se detiene en el primer byte `0x00`, que en el texto basura aparece pronto (en el ejemplo de la figura, en `temp[572]`).

La implementación de glibc usa un algoritmo más rápido (Two-Way) que devuelve el mismo resultado. Con la llave correcta, la frase se encuentra en el byte 897, donde comienza `los mismos del proyecto 1 `, y `strstr` devuelve `temp + 897`.

Reparto del espacio de llaves y búsqueda con MPI

Reparto entre N procesos

El espacio de llaves [0, `upper`) se reparte entre N procesos de dos maneras. En el modo **bloques** (el del programa original), con R = `upper` / N (división entera), el proceso `id` recorre [`id`·R, (`id`+1)·R − 1] y el último llega hasta `upper` − 1. En el modo **intercalado** el proceso `id` prueba `id`, `id`+N, `id`+2N, … Ambos modos prueban cada llave exactamente una vez; difieren en el orden en que exploran el espacio, y de ese orden depende cuándo se encuentra una llave dada.

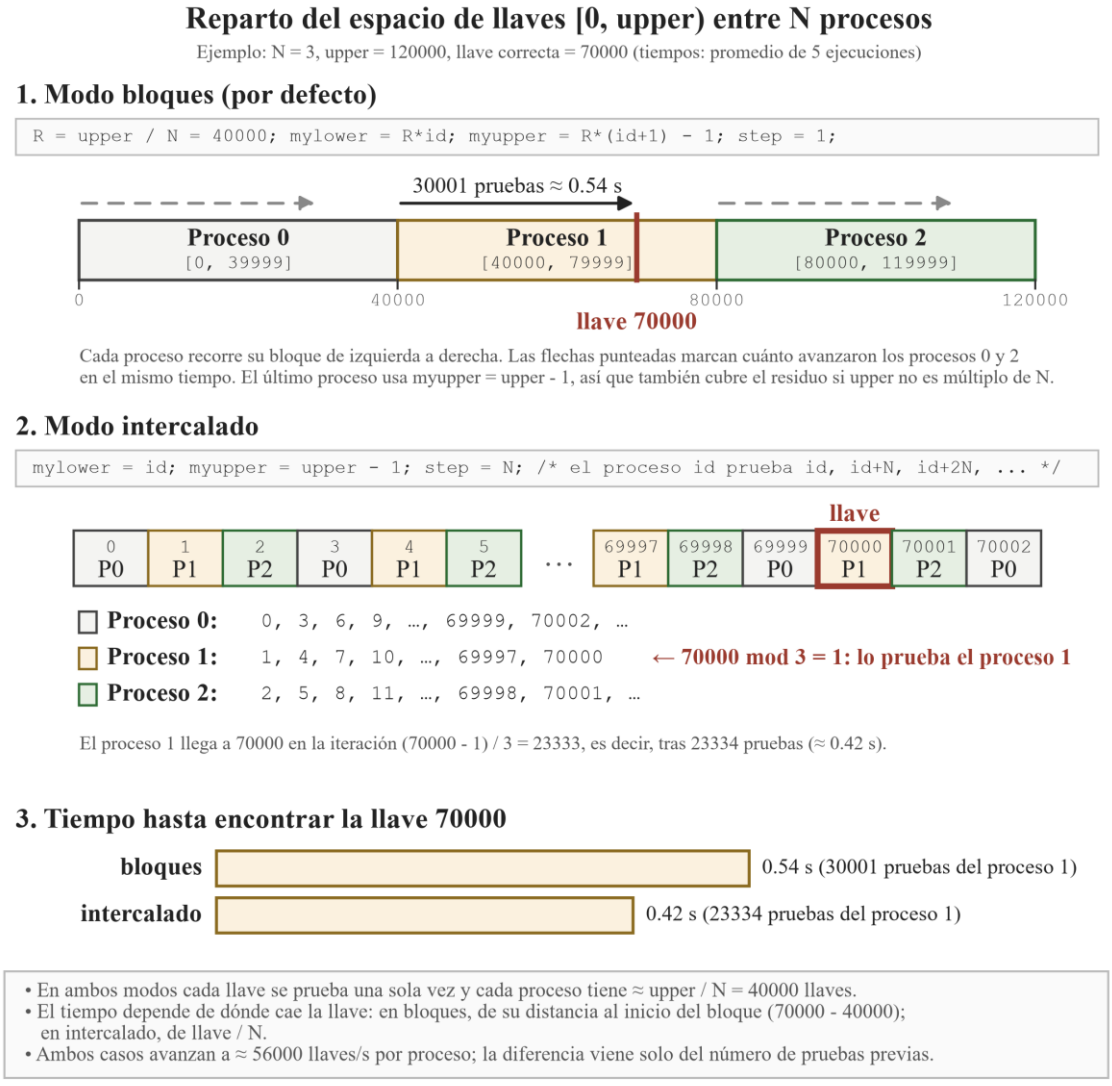


Figura 7. Reparto del espacio de llaves entre N procesos (ejemplo con N = 3, upper = 120 000 y llave 70 000).

Búsqueda y aviso de terminación

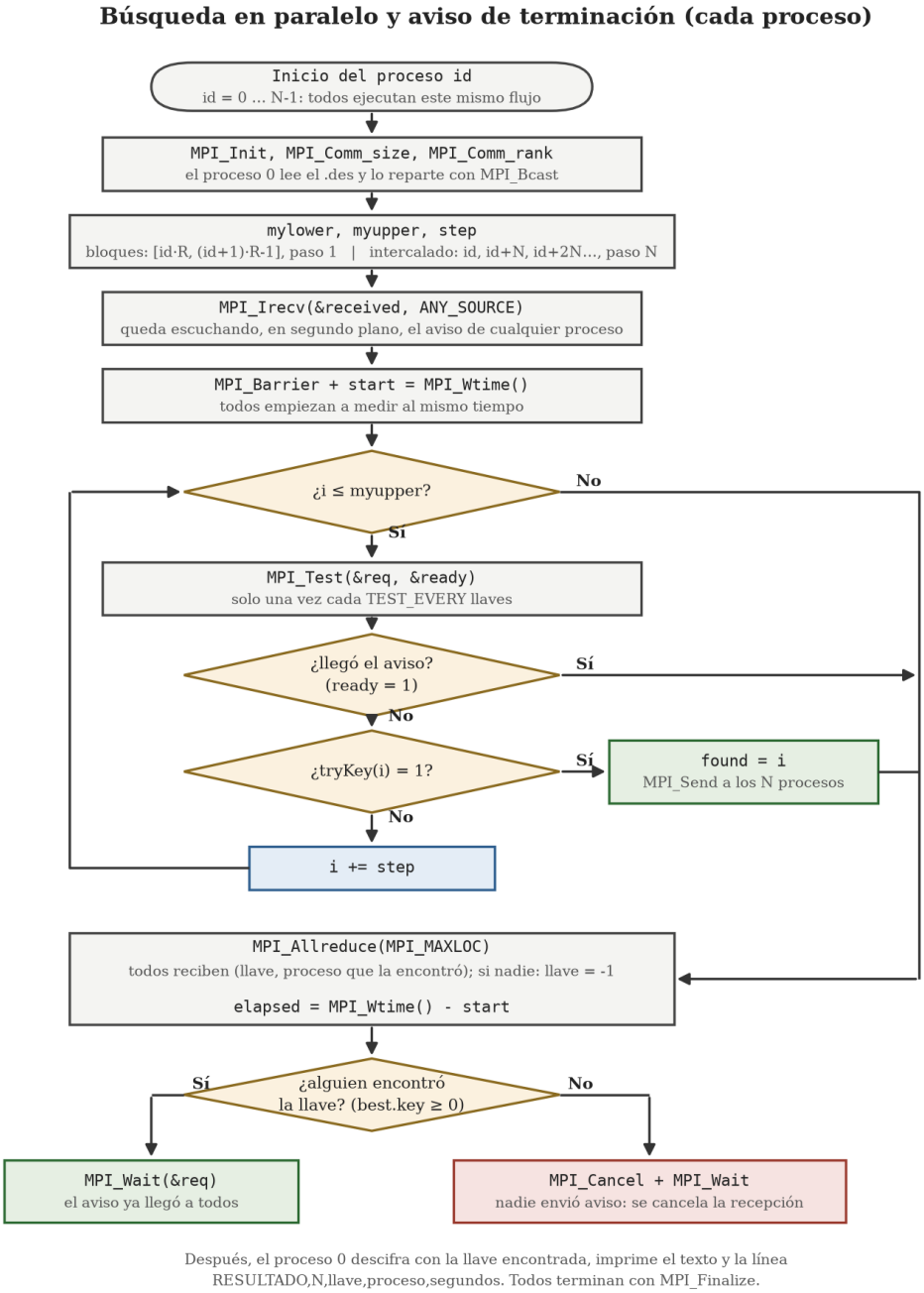


Figura 8. Búsqueda en paralelo y aviso de terminación (flujo de cada proceso).

Todos los procesos ejecutan el mismo flujo (MPI Forum, 2021):

1. El proceso 0 lee el archivo cifrado y lo reparte con `MPI_Bcast`. Cada proceso calcula su rango y publica una recepción no bloqueante con `MPI_Irecv`, que queda escuchando el aviso de cualquier otro proceso.
2. `MPI_Barrier` sincroniza a todos y se toma el tiempo inicial con `MPI_Wtime`, de modo que todos empiezan a medir al mismo tiempo.

3. En el ciclo, cada `TEST_EVERY` llaves se consulta `MPI_Test`; si el aviso ya llegó, el proceso abandona la búsqueda. Si `tryKey` devuelve 1, el proceso envía la llave a los N procesos con `MPI_Send` y sale del ciclo.
4. Al salir del ciclo se ejecuta `MPI_Allreduce` con `MPI_MAXLOC` sobre el par (llave, proceso). Así todos saben si alguien encontró la llave y quién fue, incluso cuando la llave no está en el rango y nadie envió aviso; sin este paso el programa original se quedaba esperando. Aquí se detiene el cronómetro.
5. Si hubo llave, el aviso ya está en camino y `MPI_Wait` lo recibe; si no la hubo, se cancela la recepción pendiente con `MPI_Cancel` y `MPI_Wait`.
6. El proceso 0 descifra con la llave encontrada, imprime el texto y la línea `RESULTADO`.

`TEST_EVERY` controla cada cuántas llaves se revisa el aviso. Con 1 (valor por defecto) se revisa en cada llave; con valores mayores se hacen menos consultas, a costa de que un proceso siga probando hasta `TEST_EVERY` − 1 llaves después de que otro avisó.

Mediciones de tiempo y speedup

Metodología

- **Espacio de búsqueda reducido:** `upper` = 2^{22} = 4 194 304. Recorrer las 2^{56} llaves no es posible, así que las llaves de prueba se eligen dentro de [0, `upper`). Lo que importa para el análisis es la posición de la llave dentro del bloque de cada proceso, y eso se observa igual en un espacio pequeño.
- **Llaves:** cuatro llaves en posiciones distintas (Tabla 3). Cada una se cifra con `des/encrypt_file` antes de medir.
- **Procesos y repartos:** N = 1, 2, 3 y 4, con los dos repartos (`bloques` e `intercalado`). Con N = 1 los dos repartos son el mismo algoritmo.
- **Repeticiones:** 5 por cada combinación de llave, reparto y N (4 × 2 × 4 × 5 = 160 ejecuciones). Se reporta el promedio y la desviación estándar muestral.
- **Cronómetro:** `MPI_Wtime` con `MPI_Barrier` al inicio; el tiempo se toma cuando todos los procesos ya conocen el resultado (`MPI_Allreduce`). Las repeticiones se alternan entre casos para repartir entre todos los cambios de carga de la máquina.
- **Speedup y eficiencia:** S = T(1) / T(N), donde T(1) es el tiempo promedio con un proceso y el mismo reparto; la eficiencia es S / N.
- **Condiciones:** equipo y software descritos en la sección «Compilación y ejecución de bruteforce_.c»; con N ≤ 4 no hay más procesos que CPU asignados a WSL.

Tabla 3. Llaves de prueba (upper = 2^{22} = 4 194 304).

Llave	Posición	Comportamiento esperado con bloques
123 456	Dentro del bloque 0 para cualquier N ≤ 4	S ≈ 1, el proceso 0 hace el mismo trabajo que en secuencial
2 097 151	upper/2 − 1: último elemento del bloque 0 con N = 2	S ≈ 1 con N = 2; otros valores con N = 3 y 4
2 097 153	upper/2 + 1: segundo elemento del bloque 1 con N = 2	Speedup superlineal con N = 2 y N = 4
4 194 303	upper − 1: última llave del rango	S ≈ N: todos los bloques miden lo mismo

Resultados

La Tabla 4 muestra el tiempo promedio (± desviación estándar) y el speedup de cada combinación. Las Figuras 9 y 10 los presentan por llave, con los dos repartos en cada panel.

Tabla 4. Tiempo promedio ± desviación estándar (s) y speedup, 5 repeticiones.

Llave	N	Bloques: tiempo (s)	Bloques: S	Intercalado: tiempo (s)	Intercalado: S
123 456	1	2.18 ± 0.025	1.00	2.14 ± 0.010	1.00
123 456	2	2.15 ± 0.011	1.01	1.08 ± 0.019	1.98
123 456	3	2.18 ± 0.002	1.00	0.73 ± 0.002	2.94
123 456	4	2.20 ± 0.011	0.99	0.55 ± 0.015	3.87
2 097 151	1	36.40 ± 0.18	1.00	36.48 ± 0.15	1.00
2 097 151	2	36.54 ± 0.12	1.00	18.26 ± 0.12	2.00
2 097 151	3	12.36 ± 0.016	2.94	12.39 ± 0.047	2.94
2 097 151	4	18.60 ± 0.065	1.96	9.30 ± 0.077	3.92
2 097 153	1	36.60 ± 0.42	1.00	36.54 ± 0.30	1.00

Llave	N	Bloques: tiempo (s)	Bloques: S	Intercalado: tiempo (s)	Intercalado: S
2 097 153	2	0.000066 ± < 0.001	551 265	18.23 ± 0.054	2.00
2 097 153	3	12.41 ± 0.035	2.95	12.36 ± 0.025	2.96
2 097 153	4	0.000075 ± < 0.001	485 464	9.28 ± 0.028	3.94
4 194 303	1	73.41 ± 0.76	1.00	72.78 ± 0.19	1.00
4 194 303	2	36.83 ± 0.53	1.99	36.48 ± 0.25	2.00
4 194 303	3	24.90 ± 0.37	2.95	24.77 ± 0.099	2.94
4 194 303	4	18.65 ± 0.21	3.94	18.58 ± 0.061	3.92

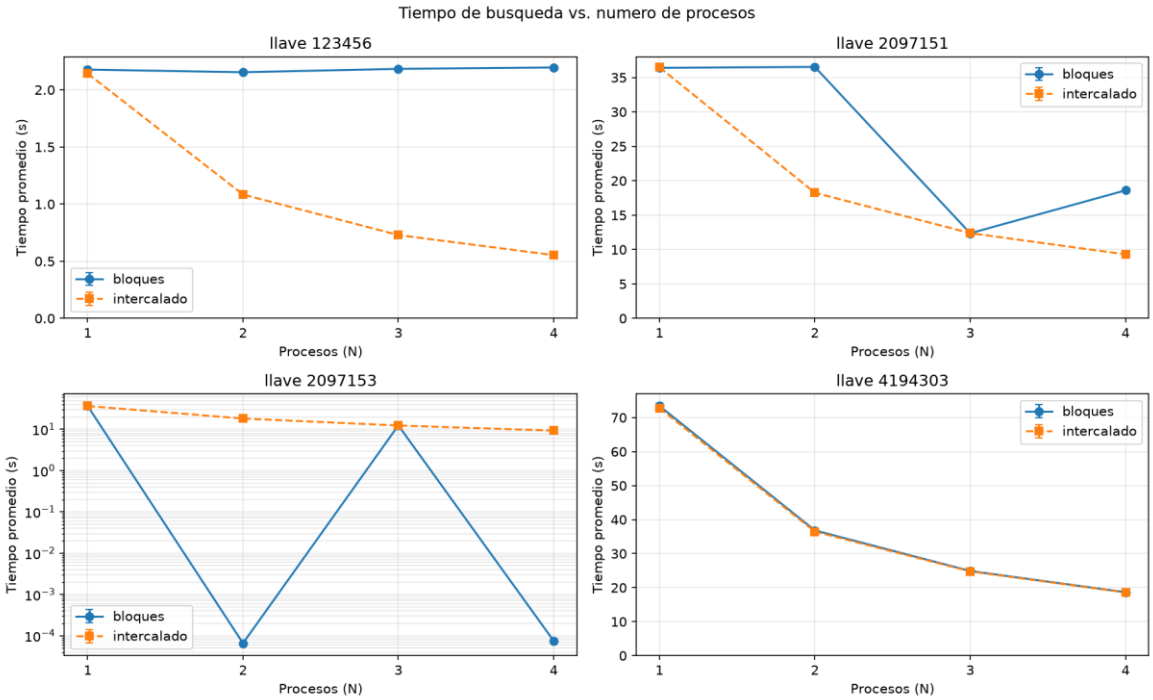


Figura 9. Tiempo promedio de búsqueda según el número de procesos, por llave y reparto.

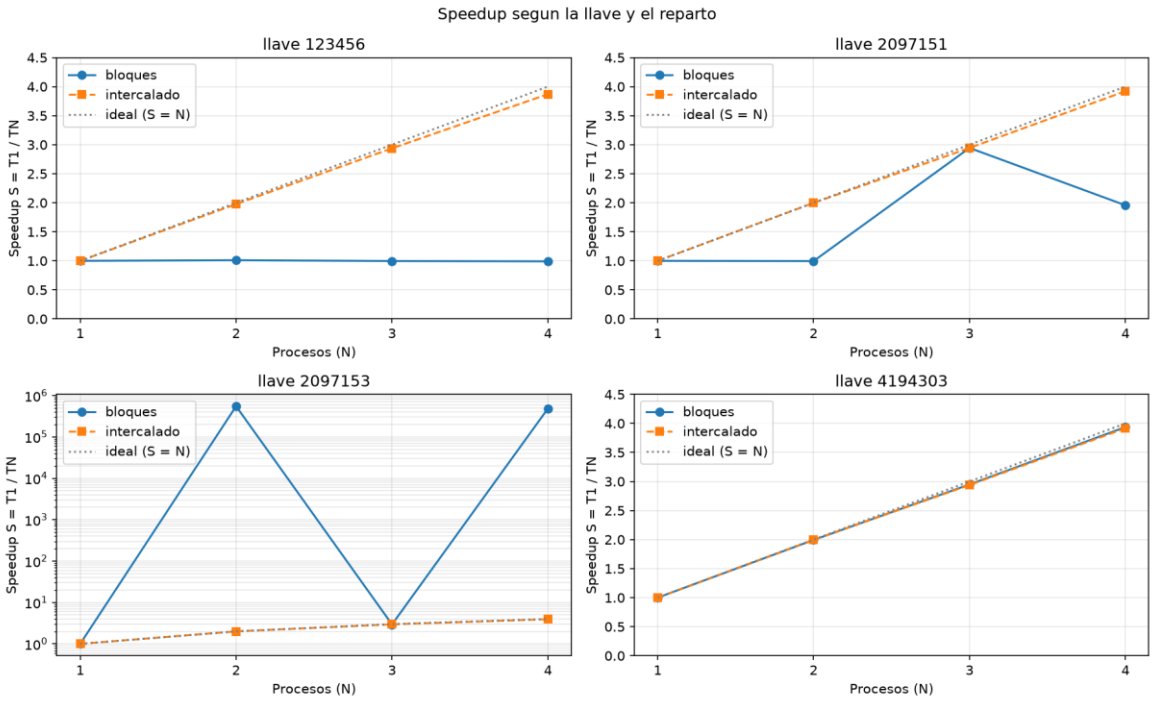


Figura 10. Speedup S = T(1) / T(N) según el número de procesos, por llave y reparto. Línea punteada gris: ideal (S = N).

Costo de MPI_Test

Con `TEST_EVERY` = 1 cada proceso llama a `MPI_Test` antes de cada llave; con 1 000, una vez cada 1 000. Se comparó con la llave 1 000 000 (que cae en el bloque 0 con N = 1 y con N = 4) y 5 repeticiones por variante:

Tabla 5. Tiempo (s) con TEST_EVERY = 1 y TEST_EVERY = 1000 (llave 1 000 000, bloques).

N	TEST_EVERY = 1	TEST_EVERY = 1000	Diferencia
1	17.423 ± 0.083	17.417 ± 0.140	-0.03 %
4	17.708 ± 0.033	17.749 ± 0.087	+0.23 %

La diferencia entre las dos versiones es del orden de la desviación estándar, así que no se puede distinguir del ruido; como cota, revisar el aviso en cada llave cuesta menos del 0.3 % del tiempo total. Es razonable: un intento cuesta ≈ 17.5 µs (312 bloques DES), mucho más que consultar si llegó un mensaje. Se deja `TEST_EVERY` = 1. Con N = 4 el tiempo no baja (S ≈ 0.98) porque la llave está en el bloque 0, igual que en la llave 123 456 de la Tabla 4.

Impacto de la llave en el rendimiento

Qué determina el tiempo

El tiempo de la búsqueda lo marca el proceso que posee la llave: los demás se detienen en cuanto les llega el aviso y no agregan tiempo. Si ese proceso necesita `t` intentos y cada intento cuesta c ≈ 17.5 µs, entonces **T(N) ≈ t · c**. Solo hay que saber cuántos intentos le tocan al dueño de la llave k (Tabla 6).

Tabla 6. Proceso dueño de la llave k e intentos que realiza.

Reparto	Proceso dueño de k	Intentos
Un proceso (N = 1)	0	k + 1
Bloques, R = upper / N	p = mín(k ÷ R, N − 1), con ÷ división entera	k − p·R + 1
Intercalado	p = k mod N	(k ÷ N) + 1, con ÷ división entera

Reparto por bloques: el speedup depende de dónde cae la llave

Con bloques, S = (k + 1) / (k − p·R + 1), y eso produce tres comportamientos muy distintos:

1. **Llave en el bloque 0 (p = 0): S ≈ 1 sin importar N.** El proceso 0 hace los mismos intentos que en secuencial y los demás buscan donde la llave no está. Es el caso de la llave 123 456: el tiempo se queda en ≈ 2.2 s y S ≈ 1.0 con 2, 3 y 4 procesos. Agregar procesos solo gasta CPU.
2. **Llave al inicio de un bloque (p ≥ 1, k ≈ p·R): speedup enorme, «superlineal».** Con la llave 2 097 153 y N = 2 o N = 4 la llave es el segundo elemento de un bloque: el dueño la encuentra en 2 intentos (≈ 70 µs) y T(1) es 36.6 s, así que S ≈ $5 \times 10^5 \gg$ N.
3. **Llave al final de un bloque: poca ganancia, y no es monótona en N.** La llave 2 097 151 es el último elemento del bloque 0 con N = 2 (S = 1.00), pero con N = 3 cae a mitad del bloque 1 (S = 2.94) y con N = 4 es el último del bloque 1 (S = 1.96). Más procesos no garantiza más speedup: depende de cómo se parta el rango.

Solo cuando la llave está al final del último bloque (4 194 303) todos los bloques miden lo mismo y el dueño recorre uno completo: S ≈ N (3.94 con N = 4, eficiencia 98 %). Es el único caso en que el reparto por bloques se comporta como el ideal.

Por qué el speedup superlineal es aparente

Un S > N no significa que el algoritmo paralelo sea más eficiente. T(1) se mide con el orden secuencial 0, 1, 2, …, y la versión paralela simplemente empieza a buscar en lugares distintos, uno de los cuales coincide con la llave. Si el programa secuencial también empezara en el bloque 1, tardaría igual. Se está comparando el orden de búsqueda, no la capacidad de cómputo.

Si la llave se eligiera al azar en [0, `upper`), el tiempo esperado sería el mismo con los dos repartos (≈ `upper` / 2N intentos). La diferencia está en la **consistencia**: con bloques, el speedup de una llave concreta puede ser 1, N o 10^5; con intercalado es siempre ≈ N.

Reparto intercalado: speedup consistente

Con intercalado todos los procesos avanzan por todo el rango a la vez, así que el que posee k la alcanza tras ≈ k / N intentos y S ≈ N para cualquier llave. Las mediciones lo confirman: con las cuatro llaves y N = 4 el speedup está entre 3.87 y 3.94 (eficiencia de 97 % a 98 %). La pérdida respecto al ideal es menor al 4 % y es mayor justo en la corrida más corta (llave 123 456, 0.55 s con N = 4); probablemente se deba a costos fijos de sincronización y cierre que pesan más cuando la corrida dura poco, pero no se midió por separado.

¿Qué hace que encontrar la llave sea más rápido o más lento?

- **Más lento:** que la llave esté lejos del inicio del bloque de su dueño (al final del bloque) o que caiga en el bloque del proceso 0, donde el paralelismo no ayuda porque el proceso 0 recorre lo mismo que en secuencial. Una llave pequeña es siempre el peor caso del reparto por bloques.
- **Más rápido:** que la llave esté cerca del inicio de un bloque de un proceso distinto del 0, o que el reparto sea intercalado, donde cada proceso aporta una fracción 1/N del recorrido sin importar la llave.
- **Lo que no influye:** el valor numérico de la llave por sí mismo. DES procesa todas las llaves con el mismo costo (en las cuatro llaves medidas, entre 17.4 y 17.6 µs por llave); lo que cambia es cuántas llaves se prueban antes de acertar.

Recomendación para medir

Para medir el rendimiento de un algoritmo de este tipo hay que usar varias llaves (al inicio, en medio y al final de los bloques) y no una sola. Con una sola llave se puede concluir que el programa «escala perfecto» o que «no escala nada», y ambas conclusiones serían falsas.

Validación del modelo

Las fórmulas de la Tabla 6 aciertan qué proceso encuentra la llave en las 32 combinaciones (16 de cada reparto) y predicen el tiempo con una diferencia de alrededor de 2 % o menos. La Tabla 7 compara lo predicho con lo medido para el reparto por bloques; el intercalado coincide de la misma forma. La excepción son los dos casos de microsegundos (llave 2 097 153, N = 2 y N = 4): se predicen ≈ 35 µs y se miden ≈ 70 µs, porque ahí el tiempo es casi todo costo fijo de sincronización (`MPI_Barrier` y `MPI_Allreduce`) y no intentos de llave.

Tabla 7. Tiempo predicho y medido, reparto por bloques, N = 2 a 4 (c = 17.49 µs por llave). Con N = 1 el dueño es siempre el proceso 0 y los intentos son k + 1.

Llave	N	Proceso dueño	Intentos del dueño	Predicho (s)	Medido (s)	¿Coincide el proceso?
123 456	2	0	123 457	2.16	2.15	sí
123 456	3	0	123 457	2.16	2.18	sí
123 456	4	0	123 457	2.16	2.20	sí
2 097 151	2	0	2 097 152	36.67	36.54	sí
2 097 151	3	1	699 051	12.22	12.36	sí
2 097 151	4	1	1 048 576	18.34	18.60	sí
2 097 153	2	1	2	0.000035	0.000066	sí
2 097 153	3	1	699 053	12.22	12.41	sí
2 097 153	4	2	2	0.000035	0.000075	sí
4 194 303	2	1	2 097 152	36.67	36.83	sí
4 194 303	3	2	1 398 102	24.45	24.90	sí
4 194 303	4	3	1 048 576	18.34	18.65	sí

Conclusiones

1. **La llave elegida cambia el resultado más que el número de procesos** cuando el espacio se reparte por bloques: para la misma N, el speedup observado va de 1 a más de 500 000 según la posición de la llave.
2. **El reparto intercalado elimina esa dependencia:** $S \approx N$ con cualquier llave, con una eficiencia de 97 % a 98 % hasta $N = 4$.
3. **El tiempo se explica con un modelo simple:** el dueño de la llave hace t intentos de ≈ 17.5 µs cada uno. El modelo acierta el proceso que encuentra la llave en las 32 combinaciones medidas y el tiempo con una diferencia de ≈ 2 % o menos.
4. **El costo de comunicación es despreciable** frente al cómputo: revisar el aviso con `MPI_Test` en cada llave cuesta menos del 0.3 % del tiempo total.
5. **El programa original no servía para medir:** no compilaba en un entorno moderno, cifraba solo 16 bytes, probaba mal los límites de cada bloque y podía quedarse esperando indefinidamente. Las correcciones de la Tabla 1 son necesarias para que las mediciones sean válidas.

Limitaciones. Las mediciones se hicieron con un espacio de búsqueda reducido (2^{22} llaves), con $N \leq 4$, 5 repeticiones por caso y en un solo equipo (WSL 2). Con tan pocas repeticiones, diferencias de décimas de por ciento no son concluyentes. DES y el modo ECB están obsoletos para uso real: aquí se emplean solo para estudiar el costo de la búsqueda exhaustiva y su paralelización.

Link GitHub: https://github.com/diegoanro22/CP-PR02.git

Referencias

Dworkin, M. (2001). Recommendation for Block Cipher Modes of Operation: Methods and Techniques, NIST SP 800-38A. https://doi.org/10.6028/NIST.SP.800-38A

Message Passing Interface Forum. (2021). MPI: A Message-Passing Interface Standard, Version 4.0. https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf

National Institute of Standards and Technology. (1999). Data Encryption Standard (DES), FIPS PUB 46-3. https://csrc.nist.gov/pubs/fips/46-3/final